

An Analysis of Adam optimizer

Adam optimizer

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Adam optimizer -- Part 1

History In 1951, Herbert Robbins and Sutton Monro introduced the earliest stochastic approximation methods, preceding stochastic gradient descent. Building on this work one year later, Jack Kiefer and Jacob Wolfowitz published an optimization algorithm very close to stochastic gradient descent, using central differences as an approximation of the gradient. Later in the 1950s, Frank Rosenblatt used SGD to optimize his perceptron model, demonstrating the first applicability of stochastic gradient descent to neural networks. Backpropagation was first described in 1986, with stochastic gradient descent being used to efficiently optimize parameters across neural networks with multiple hidden layers. Soon after, another improvement was developed: mini-batch gradient descent, where small batches of data are substituted for single samples. In 1997, the practical performance benefits from vectorization achievable with such small batches were first explored, paving the way for efficient optimization in machine learning. As of 2023, this mini-batch approach remains the norm for training neural networks, balancing the benefits of stochastic gradient descent with gradient descent. By the 1980s, momentum had already been introduced, and was added to SGD optimization techniques in 1986. However, these optimization techniques assumed constant hyperparameters, i.e. a fixed learning rate and momentum parameter. In the 2010s, adaptive approaches to applying SGD with a per-parameter learning rate were introduced with AdaGrad (for "Adaptive Gradient") in 2011 and RMSprop (for "Root Mean Square Propagation") in 2012. In 2014, Adam (for "Adaptive Moment Estimation") was published, applying the adaptive approaches of RMSprop to momentum; many improvements and branches of Adam were then developed such as Adadelta, Adagrad, AdamW, and Adamax. Within machine learning, approaches to optimization in 2023 are dominated by Adam-derived optimizers, TensorFlow and PyTorch, by far the most popular machine learning libraries, as of 2023 largely only include Adam-derived optimizers, as well as predecessors to Adam such as RMSprop and classic SGD. PyTorch also partially supports limited-memory BFGS, a line-search method, but only for single-device setups without parameter groups.

Adam optimizer -- Part 2

$Q(w) = \frac{1}{n} \sum_{i=1}^n Q_i(w) = \frac{1}{n} \sum_{i=1}^n (m(w; x_i) - y_i)^2$, where $m(w; x_i)$ is the predictive model (e.g., a deep neural network) the objective's structure can be exploited to estimate 2nd order information using gradients only. The resulting methods are simple and often effective

Adam optimizer -- Part 3

for $\Sigma(w) = \frac{1}{n} \sum_{i=1}^n (Q_i(w) - Q(w))^2$ where dB_t denotes the Ito-integral with respect to a Brownian motion is a more precise approximation in the sense that there exists a constant $C > 0$ such that

Adam optimizer -- Part 4

$w := w - \eta \nabla Q(w) = w - \eta \frac{1}{n} \sum_{i=1}^n \nabla Q_i(w)$.

Adam optimizer -- Part 5

Extensions and variants Many improvements on the basic stochastic gradient descent algorithm have been proposed and used. In particular, in machine learning, the need to set a learning rate (step size) has been recognized as problematic. Setting this parameter too high can cause the algorithm to diverge; setting it too low makes it slow to converge. A conceptually simple extension of stochastic gradient descent makes the learning rate a decreasing function η_t of the iteration number t , giving a learning rate schedule, so that the first iterations cause large changes in the parameters, while the later ones do only fine-tuning. Such schedules have been known since the work of MacQueen on k-means clustering. Practical guidance on choosing the step size in several variants of SGD is given by Spall.

Adam optimizer -- Part 6

$Q(w) = \sum_{i=1}^n Q_i(w) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (w_1 + w_2 x_i - y_i)^2$.

An Analysis of Adam optimizer

Adam optimizer

Adam optimizer -- Part 7

$$w_{\text{new}} := \arg \min_w \{ Q_i(w) + \frac{1}{2} \eta \|w - w_{\text{old}}\|^2 \}.$$

Adam optimizer -- Part 8

Stochastic gradient descent (often abbreviated SGD) is an iterative method for optimizing an objective function with suitable smoothness properties (e.g. differentiable or subdifferentiable). It can be regarded as a stochastic approximation of gradient descent optimization, since it replaces the actual gradient (calculated from the entire data set) by an estimate thereof (calculated from a randomly selected subset of the data). Especially in high-dimensional optimization problems this reduces the very high computational burden, achieving faster iterations in exchange for a lower convergence rate. The basic idea behind stochastic approximation can be traced back to the Robbins–Monro algorithm of the 1950s. Today, stochastic gradient descent has become an important optimization method in machine learning.

Adam optimizer -- Part 9

where $E\{\cdot\}$ denotes taking the expectation with respect to the random choice of indices in the stochastic gradient descent scheme. Since this approximation does not capture the random fluctuations around the mean behavior of stochastic gradient descent solutions to stochastic differential equations (SDEs) have been proposed as limiting objects. More precisely, the solution to the SDE

Adam optimizer -- Part 10

Implicit updates (ISGD) As mentioned earlier, classical stochastic gradient descent is generally sensitive to learning rate η . Fast convergence requires large learning rates but this may induce numerical instability. The problem can be largely solved by considering implicit updates whereby the stochastic gradient is evaluated at the next iterate rather than the current one:

Adam optimizer -- Part 11

Approximations in continuous time For small learning rate η stochastic gradient descent $(w_n)_{n \in \mathbb{N}_0}$ can be viewed as a discretization of the gradient flow ODE